

bcrypt

Hash de senhas que é lento de propósito

```
$2b$12$R9h/cIPz0gi.URNNX3kh20PST9/PgBkqquzi.Ss7KIUg02t0jWMUW
```

Isabela Sousa & Paulo Victor Almeida

Como guardar senhas num banco de dados?

Texto puro

Vazou o banco, vazaram todas as senhas.

SHA-256 / MD5

Feitos para serem RÁPIDOS: uma GPU testa bilhões de palpites por segundo.

bcrypt

Feito para ser LENTO e ajustável: cada palpite custa caro para o atacante.

Ideia central: o servidor nunca guarda a senha — só um hash difícil de calcular e impossível de reverter.

ORIGEM

O que é o bcrypt?

Função de hash de senhas criada por **Niels Provos e David Mazières** (USENIX, 1999) para o OpenBSD, baseada na cifra de bloco **Blowfish** (Schneier, 1993).

128 bits

Sal aleatório

Único por senha, guardado dentro do próprio hash.

2^{custo}

Fator de custo

Cada +1 dobra o tempo. Hoje: custo 12 ou mais.

72 bytes

Senha máxima

O que passar disso é ignorado.

Um hash de 60 caracteres

\$2b\$12\$R9h/cIPz0gi.URNNX3kh20PST9/PgBkqquzi.Ss7KIUg02t0jWMUW

\$2b\$

Versão do algoritmo.

12

Custo: $2^{12} = 4096$
iterações.

22 chars

Sal (16 bytes) em
base64 do bcrypt.

31 chars

Hash (23 bytes) em
base64.

Tudo que a verificação precisa (versão, custo e sal) já está no hash.

A cifra Blowfish

- Bloco de 64 bits (duas metades E e D) · rede de Feistel com **16 rodadas**
- Vetor **P** (18 subchaves) + 4 caixas **S** de 256 palavras (4 KB)
- Valores iniciais = dígitos hexadecimais de π ($P[0] = 243F6A88$): sem backdoor escondida

```
F(x) = ((S0[a] + S1[b]) XOR S2[c]) + S3[d]      a,b,c,d = bytes de x
rodada:  E ^= P[i];  D ^= F(E);  troca(E, D)
```

Por que importa: preparar a chave (reescrever P e S) custa 521 cifragens — o bcrypt explora isso.

Senha e estado inicial

Passos 1 e 2 do algoritmo

```
chave = utf8(senha) + 0x00          (máx. 72 bytes)
estado = dígitos de  $\pi$  (P e 4 caixas S)
```

O que faz: transforma a senha em bytes (com terminador `\0`) e carrega o Blowfish com os valores de π .

Por que importa: o ponto de partida é público; a senha e o sal é que tornam o estado único.

O coração do bcrypt

```
estado = ExpandKey(estado, sal, chave)
repita 2^custo vezes:
    estado = ExpandKey(estado, 0, chave)
    estado = ExpandKey(estado, 0, sal)
```

ExpandKey: $P[i] \wedge = \text{chave (repetida)} \cdot \text{cifra um bloco zerado}$ e usa o resultado para reescrever P e as 4 caixas S (521 cifragens).

Por que importa: cada iteração depende da anterior, então não dá para paralelizar nem pular etapas. O custo está AQUI.

Cifrar o texto fixo 64 vezes

```
texto = "OrpheanBeholderScryDoubt"      (24 bytes = 3 blocos)
repita 64 vezes: cifrar cada bloco com o estado final
hash = primeiros 23 bytes → base64 do bcrypt
```

O que faz: usa o estado derivado da senha para cifrar um texto conhecido; o resultado é o hash.

Por que importa: sem a senha certa o estado é outro e o resultado muda totalmente.

CUSTO

Cada +1 dobra o esforço

Medido na nossa implementação (cifragens Blowfish por hash)

Custo	Iterações (2^C)	Cifragens Blowfish	Razão
4	16	17.385	—
5	32	34.057	×1,96
6	64	67.401	×1,98
7	128	134.089	×1,99
12	4096	≈ 4,3 milhões	recomendado hoje

Conta: $(1 + 2 \cdot 2^C) \times 521 + 192$ cifragens.

Mesma senha, hashes diferentes

```
"senha123" + sal1 → $2b$04$7k1JXFsNwrvVC54aKWYw.eN4.Lpqg4MIWkD4M1mMaz0YIwO6JGLfW  
"senha123" + sal2 → $2b$04$zIz5s2MbaMaBMpY180fijefKBraB/5YwkG4YUJoL793aXJNSDSkb.
```

- Rainbow tables ficam inúteis
- Duas contas com a mesma senha não revelam isso no banco
- O atacante precisa atacar cada usuário separadamente

Atenção: o sal não é segredo — ele fica dentro do hash. O que protege é o custo.

VERIFICAÇÃO

Como o login confere a senha

1 · Ler

Extrai versão, custo e sal do hash guardado.

2 · Recalcular

Roda o bcrypt com a senha digitada, o mesmo custo e o mesmo sal.

3 · Comparar

Compara os 23 bytes em **tempo constante** (evita ataque de temporização).

Não existe “descriptografar”: o hash não é reversível.

LIMITAÇÕES

Pontos de atenção

Limite de 72 bytes

Senhas que diferem só depois do byte 72 geram o MESMO hash (mostramos na demo). Solução: pré-hash.

Pouca memória

Usa ~4 KB: melhor que SHA contra GPU, mas inferior a scrypt e Argon2 (memory-hard).

Senha fraca

bcrypt encarece cada palpite, mas "123456" cai em qualquer dicionário.

Algoritmo	Ano	Memória
bcrypt	1999	~4 KB fixa
scrypt	2009	configurável
Argon2id	2015	configurável (OWASP)

IMPLEMENTAÇÃO

100% nativo — sem imports

- Dígitos de π calculados no código (fórmula de Machin com inteiros grandes)
- Blowfish + Eksblowfish + base64 do bcrypt escritos do zero
- PRNG próprio (SplitMix64) no lugar de `os.urandom`
- Comparação em tempo constante no lugar de `hmac.compare_digest`
- Cronômetro nativo: conta cifragens Blowfish no lugar de `time`

```
python bcrypt_completo.py
```

Prova: hashes idênticos aos vetores oficiais do bcrypt (Parte 3 da demo).

CONCLUSÃO

Lento de propósito, ajustável com o tempo

- Sal aleatório + custo configurável = proteção contra rainbow tables e força bruta
- Base sólida: Blowfish e a expansão de chave custosa (Eksblowfish)
- Hoje: use custo ≥ 12 — ou Argon2id em sistemas novos

Obrigado! Perguntas?